PFE DEVSECOPS + AIOPS · CHAPITRE D'ÉVALUATION

Évaluation par injection de pannes réelles

Campagne du 13/10/2026, de 01:20 à 07:47 UTC (6.4 h) : 14 pannes provoquées sur l'infrastructure de production du projet (deux VPS), une à la fois, séparées par des périodes calmes. Pour chaque panne : délai de détection par l'IA et par les seuils classiques, alerte, décision de l'agent de remédiation et délai de rétablissement.

10/11	**92 %**	**8/9**	**2 min 40 s**
Pannes vues par l'IA délai médian 62 s	Précision de l'IA 1 fausse alerte en 6.4 h	Réparées par l'agent décisions justes 14/14	Rétablissement médian réaction de l'agent 18 s

1. Protocole

La campagne est pilotée par `eval/campagne.py`. Avant chaque panne, l'état doit être calme : modèle d'IA prêt, aucune maintenance annoncée, aucune anomalie ni alerte en cours, hors créneaux des sauvegardes, et au moins 35 min depuis une panne appelant la même action (pause de 30 min de l'agent). La panne est ensuite injectée par SSH et horodatée avec l'horloge de Prometheus. Chaque détecteur est observé, la panne est réparée par l'agent ou annulée par le script à la fin de sa durée, puis le système doit revenir au calme ; une période calme de 8 min sépare deux pannes et sert à compter les fausses alertes. Les délais sont relus dans l'historique Prometheus au pas de 5 s.

Pannes injectées

Panne	Moyen	Détection attendue	Action attendue de l'agent
Collecteur cAdvisor arrêté	docker stop	alerte de disponibilité	redémarrer le collecteur
node-exporter arrêté	docker stop	alerte « nœud injoignable »	redémarrer le collecteur
Moteur AIOps arrêté	docker stop	alerte « moteur arrêté »	redémarrer le moteur
SonarQube hors build (+2 Go)	docker start	IA (CPU ou RAM du serveur)	arrêter SonarQube
Surcharge CPU (3 cœurs sur 4)	conteneur de calcul	IA (cpu_host)	arrêter SonarQube (rien à faire)
Pression mémoire (+2 Go)	tmpfs	IA (ram_host)	arrêter SonarQube (rien à faire)
Boucle CPU dans l'API	processus dans le pod	IA (cpu_server)	redémarrage progressif de l'API
Fuite mémoire dans l'API (+150 Mo)	processus dans le pod	IA (ram_server_mb)	redémarrage progressif de l'API
Fichier de 6 Go	fallocate	IA (disk_host)	aucune (nettoyage désactivé)
Pod client à 0	kubectl scale	IA ; remise par Argo CD	aucune

Indicateurs : **rappel** = pannes détectées / pannes où le détecteur est attendu ; **précision** = épisodes d'anomalie liés à une panne / tous les épisodes ; **délai** depuis l'injection ; **décision correcte** = action choisie par l'agent égale à l'action attendue, même si un garde-fou l'a bloquée ; **rétablissement** = début de la panne → panne disparue.

2. Résultats

2.1 Détection

Détecteur	Pannes détectées	Délai médian	Min – max
Moteur AIOps (z-score ou Isolation Forest)	91 % (10/11)	62 s	50 s – 75 s
z-score robuste seul	91 % (10/11)	62 s	50 s – 75 s
Isolation Forest seul	0 % (0/11)	—	—
Alerte AnomalieAIOps (après 1 min)	91 % (10/11)	2 min 12 s	2 min 00 s – 2 min 25 s
Seuils fixes (CPU > 80 %, RAM > 85 %, disque > 85 %)	0 % (0/11)	—	—
Alertes de disponibilité	100 % (3/3)	2 min 25 s	2 min 15 s – 2 min 35 s

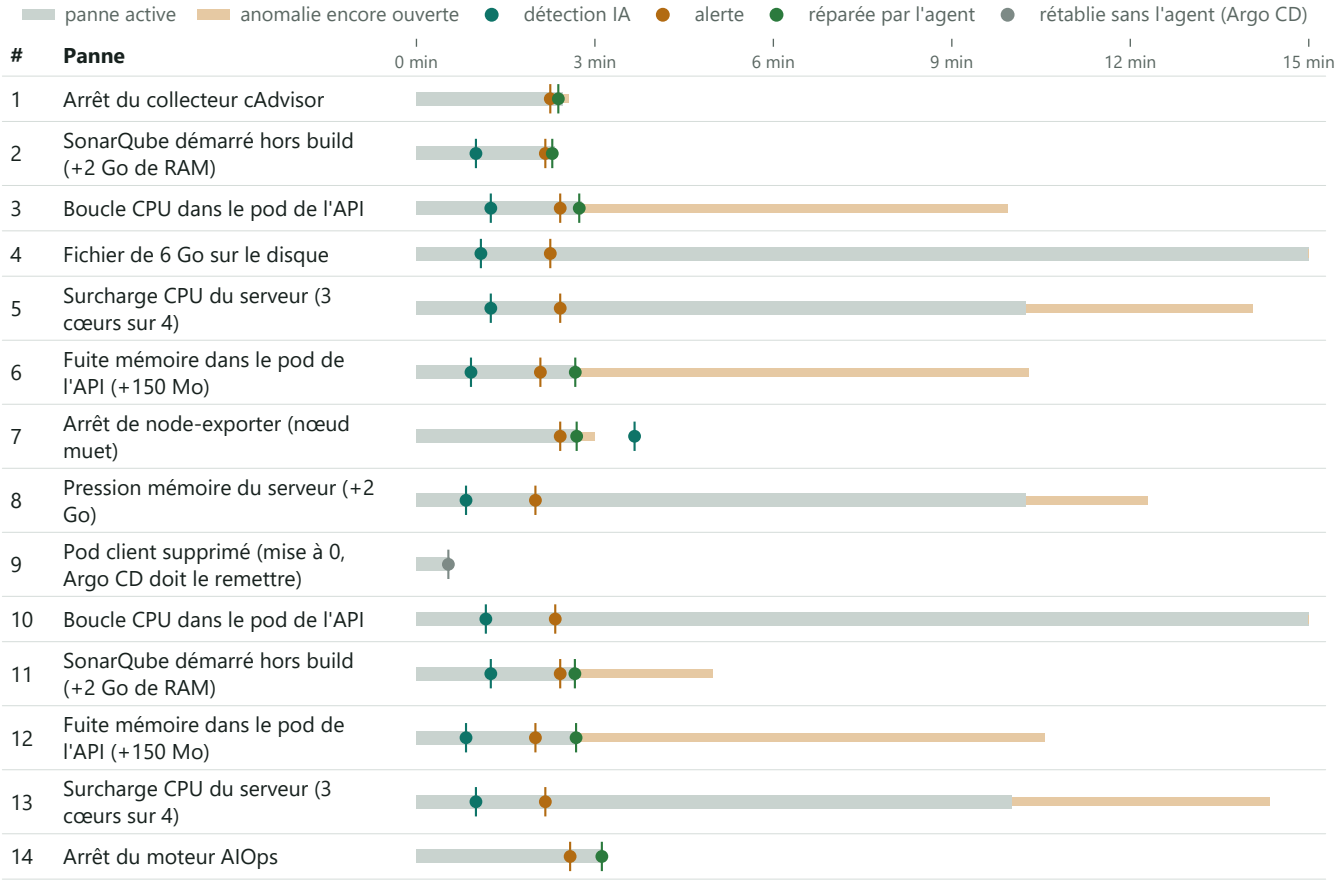
Précision du moteur AIOps : 92 % (12 épisodes liés à une panne sur 13) ; fausses alertes hors pannes : app-1 03:56–03:58, soit 0.16 par heure.

Diagnostic : la variable désignée par l'IA comme responsable est la bonne dans 10 cas sur 10.

2.2 Agent de remédiation

Indicateur	Résultat
Décisions correctes (action choisie = action attendue)	14/14
Pannes réparables effectivement réparées par l'agent	8/9
Délai de rétablissement (médiane)	2 min 40 s
Réaction de l'agent après l'alerte (médiane)	18 s

2.3 Chronogramme de chaque panne



2.4 Détail des injections

Délais en secondes depuis l'injection. « Rétablie » : panne disparue (par l'agent, ou par Argo CD pour la panne 9).

#	Panne	Heure	IA	Alerte	Variable IA	Décision de l'agent	OK	Rétablie
1	Arrêt du collecteur cAdvisor	01:21	—	135 s	—	restart-container cadvisor (exécutée)	oui	143 s
2	SonarQube démarré hors build (+2 Go de RAM)	01:32	60 s	130 s	cpu_host	stop-sonarqube (exécutée)	oui	137 s
3	Boucle CPU dans le pod de l'API	02:04	75 s	145 s	cpu_server	rollout-restart server (exécutée)	oui	164 s
4	Fichier de 6 Go sur le disque	03:06	65 s	135 s	disk_host	aucune action	oui	—
5	Surcharge CPU du serveur (3 cœurs sur 4)	04:09	75 s	145 s	cpu_host	stop-sonarqube (exécutée)	oui	—
6	Fuite mémoire dans le pod de l'API (+150 Mo)	04:56	55 s	125 s	ram_server_mb	rollout-restart server (exécutée)	oui	160 s
7	Arrêt de node-exporter (nœud muet)	05:14	220 s	145 s	—	restart-container node-exporter (exécutée)	oui	162 s
8	Pression mémoire du serveur (+2 Go)	05:25	50 s	120 s	ram_host	stop-sonarqube (bloquée : pause)	oui	—
9	Pod client supprimé (mise à 0, Argo CD doit le remettre)	05:45	—	—	—	aucune action	oui	32 s
10	Boucle CPU dans le pod de l'API	05:54	70 s	140 s	cpu_server	rollout-restart server (bloquée : quota)	oui	—
11	SonarQube démarré hors build (+2 Go de RAM)	06:21	75 s	145 s	cpu_host	stop-sonarqube (exécutée)	oui	160 s
12	Fuite mémoire dans le pod de l'API (+150 Mo)	06:53	50 s	120 s	ram_server_mb	rollout-restart server (exécutée)	oui	161 s
13	Surcharge CPU du serveur (3 cœurs sur 4)	07:12	60 s	130 s	cpu_host	stop-sonarqube (exécutée)	oui	—
14	Arrêt du moteur AIOps	07:34	—	155 s	—	restart-container mon-aiops-engine (exécutée)	oui	187 s

3. Discussion

- **L'IA détecte ce que les seuils classiques ne voient pas.** Aucune panne n'a franchi les seuils fixes (CPU > 80 %, RAM > 85 %, disque > 85 %), alors que le moteur AIOps en a vu 10 sur 11 en 62 s (médiane), en désignant la bonne variable. Seule la suppression du pod client n'a pas été vue : Argo CD l'a remis en une trentaine de secondes, avant la fenêtre d'observation du modèle.
- **Seul le z-score robuste a détecté.** L'Isolation Forest, appris sur deux heures d'historique puis suspendu après un changement de régime, n'a rien signalé. Sa contribution reste à mesurer après 24 h d'historique et calibration.
- **L'agent prend des décisions justes et répare vite.** 14 décisions justes sur 14 et 8 pannes réparables sur 9 réparées, avec un rétablissement médian de 2 min 40 s, dont une réaction de 18 s après l'alerte. Le délai est dominé par le délai de confirmation des alertes (1 à 2 min), choisi pour éviter les fausses alertes.
- **La réparation manquée l'a été par prudence.** Le quota de 3 actions par heure était atteint, en partie à cause d'une action sans effet déclenchée par une anomalie apparue juste après le redémarrage d'un collecteur. Correction appliquée : une action sans effet ne compte plus dans le quota.
- **Un effet de bord a été identifié et corrigé.** Le démarrage de SonarQube a fait libérer par le noyau le cache mémoire de l'API : sa RAM a baissé durablement et l'anomalie est restée ouverte 30 min, jusqu'à ce que le moteur accepte ce nouveau régime. L'agent ne traite plus une baisse comme une surcharge.

4. Limites et suites

- Effectif réduit (14 pannes, 10 types) : les pourcentages sont indicatifs, pas statistiquement robustes.

- Modèle appris sur quelques heures, de nuit, sans calibration : refaire la campagne après 24 h calmes et `aiops.calibrate` pour comparer avant et après.
- Pannes simples et isolées : pas de pannes simultanées, pas de dégradation lente hormis le disque.
- Une panne a été interrompue par une coupure SSH du poste de pilotage ; elle est exclue des statistiques et sa fenêtre est seulement utilisée pour ne pas compter son anomalie comme fausse alerte.

Données brutes et méthode : dépôt du projet, dossier `eval/` (campagne.py, analyse.py, resultats/injections.jsonl, README.md).